



Day 4 Notes

Database Basics – Simple Explanation



★ Today = made data actually PERMANENT, using a real database. Also moved our repo to a GitHub Organization.

0 Repo Moved to an Organization

Our repo now belongs to a team account, not just my personal account. My friend can be added as a proper member.

```
git remote set-url origin https://github.com/VIRTUAL-TRYON-STORE/virtual_tryon_store.git
```

This tells our LOCAL folder: "the shared copy now lives at this new address." Both of us need to run this once on our own laptops.

1 What is a Database?

A database is organized storage that our CODE can read and write to – like a spreadsheet, but smarter.

- Table = one sheet (example: "products")
- Row = one record (one single product)
- Column = one piece of info about it (name, price, etc.)

We used SQLite – a database that lives in ONE file, no separate server needed. Great for learning and small projects.

2 Creating a Table

```
const Database = require('better-sqlite3')
const db = new Database('store.db')

db.exec(`
  CREATE TABLE IF NOT EXISTS products (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    price REAL NOT NULL
```

- new Database('store.db') → creates/opens a database file
- CREATE TABLE IF NOT EXISTS → makes the table, only if not already there
- id INTEGER PRIMARY KEY AUTOINCREMENT → auto-numbered unique ID (1, 2, 3...)
- TEXT / REAL → column types (text, and decimal numbers)

3 Adding Data (INSERT)

```
const insert = db.prepare('INSERT INTO products (name, price) VALUES
(?, ?)')
insert.run('Blue T-Shirt', 19.99)
```

The ? marks are placeholders — db.prepare makes a safe reusable template, then .run(...) fills in real values. This protects against a common attack called SQL injection.

⚠ Running the insert code again adds the SAME rows again (duplicates) — always comment it out once your data is in!

4 Reading Data Back (SELECT)

```
const products = db.prepare('SELECT * FROM products').all()
console.log(products)
```

SELECT * FROM products means "get every column, from every row, in the products table." .all() returns them as a normal JavaScript array.

Big proof: this data stayed even after closing and reopening the file — unlike React state, which resets on every page refresh!

5 Connecting the Database to Express

We replaced our Day 3 hardcoded array with a real query:

```
const Database = require('better-sqlite3')
const db = new Database('store.db')

app.get('/api/products', (req, res) => {
  const products = db.prepare('SELECT * FROM products').all()
  res.json(products)
})
```

Now React → Express → Database are all connected. React never touches the database directly — it always goes through Express.



Bug I Fixed Today

Got this error: `SqliteError: no such table: product`

Cause: I wrote `FROM product` (singular) but the table is named `products` (plural). SQL errors are almost always typos like this — check table/column names carefully!



Big takeaway:

Database = permanent, organized storage. SQLite = a simple file-based database for learning.

`CREATE TABLE` / `INSERT` / `SELECT` = the 3 basic SQL actions. Full flow now complete: React (shows data) → Express (serves data) → Database (stores data), the real architecture behind our project.

— made while building: AI Virtual Try-On E-commerce App —